UIUC CS 225 - Theory Review

Anirudh Konidala

January 2025

1 Recursive Definitions

Recursive definitions contain formulas where each term is based on one or more preceding terms

- Many recursive numerical formulas have something called a **closed form**, which is a closed expression that doesn't involve computing each term in the formula/sequence (similar to a summation)
- Deriving this closed form is an act known as **unrolling**, where we substitute a recursive definition into itself until there is an explicit pattern suggesting the closed form expression

For example with the expression,

$$\begin{aligned} T(n) &= 2T(n-1) + 3 \\ &= 2(2T(n-2) + 3) + 3 \\ &= 2(2(2T(n-3) + 3) + 3) + 3 \\ &= 2^3 T(n-3) + 2^2 \cdot 3 + 2 \cdot 3 + 3 \\ &= 2^4 T(n-4) + 2^3 \cdot 3 + 2^2 \cdot 3 + 2 \cdot 3 + 3 \\ &\dots \\ &= 2^k T(n-k) + 2^{k-1} \cdot 3 + \cdots + 2^2 \cdot 3 + 2 \cdot 3 + 3 \end{aligned}$$

we can use summation notation to represent the kth unrolling step, where

$$\begin{aligned} &= 2^k T(n-k) + 2^k - 1 \cdot 3 + ... + 2^2 \cdot 3 + 2 \cdot 3 + 3 \\ &= 2^k T(n-k) + 3(2^k - 1 + \cdots + 2^2 + 2 + 1) \\ &= 2^k T(n-k) + 3 \sum_{i=0}^{k-1} (2^i) \end{aligned}$$

The input of function T hits the base case when $n - k = 1$, or when $k = n - 1$. Recall that $T(1) = 1$ and substituting $n - 1$ value for k gives

$$\begin{aligned} T(n) &= 2^k T(n-k) + 3\sum_{i=0}^{k=1}(2^i) \\ &= 2^{n-1}T(1) + 3\sum_{i=0}^{n-2}(2^i) \\ &= 2^{n-1} + 3\sum_{k=0}^{n-2}(2^k) \\ &= 2^{n-1} + 3(2^{n-1} - 1) = 4(2^{n-1}) - 3 = 2^{n+1} - 3 \end{aligned}$$

Therefore making the closed form for function T, $2^{n+1} - 3$

1.1 Example Problem 1

Given that $T(0) = 5$, find the closed form solution of

$$T(n) = T(n-1) + 2$$

Step 1: Unroll the first couple of terms

$$\begin{aligned} T(n) &= T(n-1) + 2 \\ &= (T(n-2) + 2) + 2 \\ &= ((T(n-3) + 2) + 2) + 2 \\ &\dots \\ &= T(n-k) + 2k \end{aligned}$$

where $k = n$ and $T(n) = T(0) + 2n = 5 + 2n$

Step 2: Recall that $T(0)$ can represent any sort of numeric value, where in this case is 5. This numeric value of 5 is known as the **base case** and $2n$ is known as the **recursive case**

1.1.1 Example Problem 2

Given that $T(1) = 3$, Find the closed form solution of

$$T(n) = 2T(\frac{n}{2}) + n$$

Step 1: Unroll the first couple terms of the sequence

$$\begin{aligned} T(n) &= 2T(\frac{n}{2}) + n \\ &= 2(2T(\frac{n}{4}) + \frac{n}{2}) + n = 4T(\frac{n}{4}) + 2n \\ &= 2(2(2T(\frac{n}{8}) + \frac{n}{4}) + n) + n = 8T(\frac{n}{8}) + 3n \\ &\cdots \\ &= 2^k T(\frac{n}{2^k}) + kn \end{aligned}$$

Step 2: where $\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$ and

$$\begin{aligned} T(n) &= 2^{\log_2 n} \cdot T(1) + n \cdot \log_2 n \\ &= n \cdot T(1) + n \log_2 n \\ &= 3n + n \log_2 n \end{aligned}$$

1.2 Proofs by Induction

Induction is a technique used to prove recursive definitions/ formulas, indicating that it does match its respective closed form solution

- Includes a **base case**, **inductive hypothesis**, and **inductive step**
- **Base Case** - The starting point, where you prove that it holds true for the smallest value, typically $n = 0$ or $n = 1$
- **Inductive Hypothesis** - A assumption that the statement being proven is true for some value k
- **Inductive Step** - Prove the inductive hypothesis with the next step $k+1$

1.2.1 Example Problem 1

Prove that $1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$ for $n = 1$ **Step 1**: Let $n = 1$, then $\frac{1(1+1)}{2} = 1$, where $1 = 1$ (both sides are equal) **Step 2**: Assume the formula holds true for $k \geq 1$, where $1 + 2 + 3 + \cdots + k = \frac{k(k+1)}{2}$ **Step 3**: Prove the inductive hypothesis from b. for $k + 1$, the next recurring step

$$\begin{aligned} 1 + 2 + \cdots + k + (k+1) &= \frac{(k+1)(k+2)}{2} \\ (1 + 2 + \cdots + k) + (k+1) &= \frac{(k)(k+1)}{2} + (k+1) \\ \frac{k(k+1) + 2(k+1)}{2} &= \frac{(k+2)(k+1)}{2} \end{aligned}$$

where the inductive step does indeed hold

1.2.2 Example Problem 2

Prove $2^n \geq n + 1$

Step 1: Check if true for $n = 0$

$$2^0 \geq 0 + 1$$
$$1 \geq 1$$

The base case does hold

Step 2: Assume the statement holds for $n = k$, where

$$2^k \geq k + 1$$

Step 3: Using this assumption, prove it holds for $n = k + 1$

$$2^{k+1} \geq (k + 1) + 1$$
$$2^{k+1} \geq k + 2$$
$$2^k \cdot 2 \geq 2(k + 1) \geq k + 2$$

2 Big-O

Big-O describes how fast a function grows as its input size, n, increases. It observes the asymptomatic behavior as n gets large.

- measures the efficiency of algorithms
- compares runtime/space usage based on large inputs
- describes the worst case runtime of an algorithm
  - or the **upper bound** of an algorithm, which indicates how bad the algorithm's runtime can get

Big-O Notation	Name	Typical Algorithmic Behavior
$O(1)$	Constant Time	Accessing a single array element
$O(\log n)$	Logarithmic Time	Binary Search; algorithm halves the input per iteration
$O(n)$	Linear Time	Linear Search; looping over an array once
$O(n \log n)$	Log-Linear Time	Merge Sort, Quick Sort, Heap Sort; Divide and conquer, where problems are broken into subproblems that are solved individually
$O(n^2)$	Quadratic Time	Bubble Sort, Selection Sort, Insertion Sort; Anything involving nested for loops
$O(2^n)$	Exponential Time	Typically many recursive algorithms, including brute force (attempts all possible solutions to a problem until one satisfies the problem's conditions)
$O(n!)$	Factorial Time	Permutation-Based Solutions

Table 1: Common Big-O Time Complexities and Their Typical Behaviors

Algorithm	Big-O Notation	Description
Linear Search	$O(n)$	Checks each element one by one from beginning to end looking for the given element
Binary Search	$O(\log n)$	Takes in a sorted array and iteratively halves the array that may contain the given value
Bubble Sort	$O(n^2)$	Repeatedly steps through an array and compares adjacent elements, swapping them if they are in the wrong order
Insertion Sort	$O(n^2)$	Iteratively inserts each element of an unsorted list into its correct position in a sorted portion of the list
Selection Sort	$O(n^2)$	Sorts an array by repeatedly selecting the **smallest** element from the unsorted portion and swapping it with the first unsorted element
Merge Sort	$O(n \log n)$	Recursively divides an array/list into smaller sublists, where each sublist is sorted and then merged back together into one big list
Quick Sort	$O(n^2)$ with average case of $O(n \log n)$	Selects a pivot element from the array and partitions the other elements into two sub-arrays, based on whether they are less than or greater than the pivot
Heap Sort	$O(n \log n)$	Organizes elements in an array using a binary heap, by repeatedly swapping the root node (whether it's the maximum or minimum element for max/min heap respectively) with the last element of the heap

Table 2: Common Sorting and Searching Algorithms with their Time Complexities

3 Graphs

Graphs consist of a set of nodes (points/vertices), V, and a set of edges, E, where the graph is commonly referred to as a pair of sets (V, E).

Each edge in E joints two nodes in V, where two nodes connected by an edge are **adjacent** or **neighbors**

Edges can have **direction**, where graphs with these edges are known as **directed graphs**. This is useful when discussing relations between vertices with contexts to their edges. In contrast, edges that can be traversed in both directions are known as **undirected graphs**

3.1 Additional Graph terminology

- **Loops** - a node where an edge connects to itself
- **Multiple Edges** - two nodes are connected by multiple edges
- **Complete Graphs** - Graphs where each node is connected to every other node. They have $\frac{n(n-1)}{2}$ edges
- **Walks** - A sequence of nodes and edges, where each edge connects the node before and after it and nodes and edges can repeat
- **Paths** - Walks that **cannot repeat nodes and edges**
- **Cycles** - A path that **starts** and **ends** at the same
  - **Cyclic Graphs** consist of graphs where nodes are connected in a **cycle** (non cyclic graphs are called **acyclic**)
- **Wheels** - Cycle Graphs where there exists a **single central node** that connects to every other node in the cycle graph - **Subgraphs** - A smaller graph made from a subset of vertices and edges of a bigger graph

A graph is **simple** if it does not contain **loops** or **multiple edges**

3.2 Degree

The degree of a node v, notated as $deg(v)$, is the number of edges that are connected to v. Loops count **twice** towards the degree of a node.

Each edge contributes to two degrees of a node. Therefore, the **sum of the degrees of all nodes in a graph** is **twice** the number of edges, which can be mathematically expressed as

$$\sum_{v \in V} deg(v) = 2|E|$$

or

$$\sum_{k=1}^{n} v_k \in Vdeg(v) = 2|E|$$

This is known as the **Handshaking Theorem**

3.3 Isomorphism

Isomorphism exists when two graphs have the same structure, albeit the drawings may be different

Two graphs, G_1 and G_2 are isomorphic when there exists a bijection of

$$f : V_1 \to V_2$$

Or simply,

- The two graphs must have the same number of nodes and the same number of edges
- For any node degree k, the two graphs must have the same number of nodes of degree k. For example, they must have the same number of nodes with degree 3
- Given a subgraph of G_1, that subgraph must also exist in G_2

3.4 Connectivity

A graph is **connected** when there exists a walk between every pair of nodes in G, or simply you can get from **any node to any other node**

Sometimes removing an edge from a connected graph will completely lose its connectivity. We refer to such edge as a **cut edge** when if removed, the graph would be disconnected. Similarly, a **cut vertex** exists when removing such vertex increases the number of connected components in a graph

An **Euler Circuit** exists when there is a closed walk (circuit) that uses every edge of the graph once. This can only occur when **all degrees of the nodes are even**

This is because in order to complete the circuit, you must leave each node that you enter. This means that the graph is **connected** and therefore exists a closed walk that uses every edge of the graph once

3.5 Graph Coloring

Graph coloring is a **technique** used to assign colors to a node of a graph, where two adjacent vertices **cannot** share the same color

3.5.1 Graph Coloring Steps

1. Let node, v_1, be the node with the **greatest degree**
2. Select a color and color **all non neighboring nodes** to v_1 (i.e. an edge does not connect v_1 and another node)
3. Repeat steps 1 and 2 for all non-colored nodes until **all nodes in the graph are colored**

The number of colors it takes to color a graph, is known as the **chromatic number**, mathematically notated as $\chi(G)$

3.6 Bipartite Graphs

A graph, $G = (V, E)$ is **bipartite**, such that V can be split into two non-overlapping subsets V_1 and V_2, where every edge connects to an element of V_1 in V_2.

What this simply means is that **a graph can only be bipartite when its** $\chi(G) = 2$. This means that a graph can only be colored with **two colors**

4 Trees

Trees are undirected graphs with a special node called the **root**, where every node is connected to the root via one exact **path**

They are also **acyclic**, **connected**, and have n **vertices** with $n - 1$ **edges**

4.1 Tree Vocabulary

- **Parent** - A node that is directly above another node in a tree
- **Child** - A node that is directly below another node in a tree
- **Sibling** - Nodes that **share the same parent**
- **Leaf** - A node that has no children
- **Internal** - A node that has at least one child
- **Level** - The number of edges from the **root** to the node (the root has a level of 0)
- **Height** - The longest path from a node to a leaf (in terms of edges)
- **Ancestor** - The ancestor of a node is one that lies on the path from the root to such node
- **Descendant** - The descendant of a node is one that lies on the path from such node to a leaf node

4.2 m-ary Trees

m-ary trees are trees that can have at most m children. A common type of tree is a **Binary Search Tree**, where each node has at most 2 children.

- **Full** trees exist when each node has either 0 or m children
- **Complete** trees exist when all levels are completely filled, except for the lowest one, which is filled from the left
  - A tree is **full** and **complete** when the whole bottom level is fully populated with leaves

  - **Balanced** trees exist when the heights of its left and right subtrees **differ by at most** 1

There are $mi + 1$ nodes total in a m-ary tree. This means a binary tree has $2i + 1$ nodes total

4.3 Height and number of nodes

There is no exact formula to find the number of nodes and leaves in an m-ary tree. This is because some trees are more bushier (have more children) than others.

However, there are some patterns **(assuming that a given tree is binary)**

- The number of nodes are maximized when a tree, with height h, is full and complete is 2^h
  - where the number of nodes at level L are 2^L and the total nodes are $\sum_{L=0}^{h} 2^L$, with a closed form of $2^{h+1} - 1$